ME 469: Machine Learning and Artificial Intelligence for Robotics

Assignment 0: Filtering Algorithms

By: Tomasz Krzeminski
Date: 10/06/2025

Introduction

Robotics as a field is focused on developing robots capable of actuation, sensing and control. To achieve these goals, robots are outfitted with sensors and actuators that communicate with an onboard processor, which is responsible for performing high and low level control to allow the robot to achieve its task. Russell and Norvig (2020) discretize robot action into task planning, motion planning and control. Task planning is the aforementioned high-level planning, which commands a robot with non-specific tasks. Motion planning is the intermediary-level which provides the robot with a path to achieve the task planning goals. Finally, control is the low-level subroutine which utilizes sensor readings to command the actuators to achieve the goals above. As such, sensors and actuators are the main components that enable the robot to interact with the environment and achieve state transition. Because of the stochasticity of the physical environment however, sensors and actuators experience noise and uncertainty, which makes it difficult to accurately estimate the state of the robot. Furthermore, onboard processors are oftentimes small, and therefore less computationally powerful. This presents challenges in control and sensing as low computational power limits data streaming and processing, which can further reduce the accuracy of the state prediction.

To address these challenges, probabilistic filters are employed to account for noise and uncertainty and yield the state of a robot. According to Thurn et. al (2005, p.22), the aim of all filters is to estimate the state transition given the previous state and current measurements of the robot. The types of filters used in this state estimation are often classified as either Gaussian filters (parametric) or Non-parametric filters. Gaussian filters are unimodal, and therefore represent the state prediction as a Gaussian, and have a moments parameterization, which means they are represented by a mean and covariance (Thurn et. al, 2005, p.33). The major advantage of Gaussian filters is that they are computationally tractible, however, a major disadvantage is that they are unimodal in output, which means the state prediction is often constrained. Non-parametric filters are multi-modal, which means they have no formal constraint on the output of the posterior belief. The means by which these filters achieve the state transition prediction is by sampling a discrete number of samples which are often limited to a small representation of the action space (Thurn et. al, 2005, p.68). The major advantage of Non-parametric filters is that the posterior is unconstrained, which allows for more diverse sampling, and therefore yields more comprehensive information. The downside to these filters is that computations are often performed on discrete sets of samples, which makes the filters less computationally tractible with larger sample sizes.

$$p(x_t \mid x_{t-1},\ u_t) \tag{1}$$

$$p(z_t \mid x_t) \tag{2}$$

As was mentioned above, all filters utilize information about the previous state, and measurements to estimate the state transition. A state transition probability and measurement probability are used to take these inputs and compute probabilities which are used to compute the belief of the next state. The state transition probability, as seen in equation (1), is a probability distribution function that takes into account the previous state of the robot, as well as control commands which perturb the state (Thurn et. al, 2005, p.21). By modeling the state as a probability distribution, the model is able to account for the uncertainty and stochasticity of the state transition. The measurement probability, as seen in equation (2), is also a probability distribution, however, it only takes into account the current state of the robot. This probability accounts for the noise in the sensors that yield the measurement (Thurn et. al, 2005, p.21). The measurement probability is oftentimes utilized to correct the state transition probability.

1. Motion Model Design

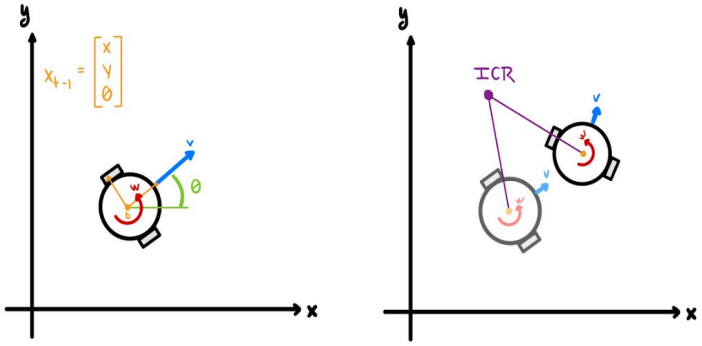


Figure 1: State Representation of Robot (left) and Instantaneous Center of Rotation for Robot (right)

$$\begin{aligned} \hat{v} &= v + sample(\alpha_1|v| + \alpha_2|w|) \\ \hat{w} &= w + sample(\alpha_3|v| + \alpha_4|w|) \\ \gamma &= sample(\alpha_5|v| + \alpha_6|w|) \end{aligned} \tag{3}$$

$$sample(b) = \frac{b}{6}\sum_{i=1}^{12} rand(-1, 1) \tag{4}$$

$$ICR = \begin{bmatrix} x_c \\ y_c \end{bmatrix} = \begin{bmatrix} x - \frac{\hat{v}}{\hat{w}}\sin\theta \\ y + \frac{\hat{v}}{\hat{w}}\cos\theta \end{bmatrix} \tag{5}$$

$$R = \begin{bmatrix} \cos(\hat{w}\cdot dt) & -\sin(\hat{w}\cdot dt) & 0 \\ \sin(\hat{w}\cdot dt) & \cos(\hat{w}\cdot dt) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{6}$$

$$X_t = R\begin{bmatrix} x - x_c \\ y - y_c \\ \theta \end{bmatrix} + \begin{bmatrix} x_c \\ y_c \\ (\hat{w}+\gamma)\cdot dt \end{bmatrix} \tag{7}$$

$$= \begin{bmatrix} \cos(\hat{w}\cdot dt) & -\sin(\hat{w}\cdot dt) & 0 \\ \sin(\hat{w}\cdot dt) & \cos(\hat{w}\cdot dt) & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x - x_c \\ y - y_c \\ \theta \end{bmatrix} + \begin{bmatrix} x_c \\ y_c \\ (\hat{w}+\gamma)\cdot dt \end{bmatrix} \tag{8}$$

$$= \begin{bmatrix} (x - x + \frac{\hat{v}}{\hat{w}}\sin\theta)\cos(w\cdot dt) - (y - y - \frac{\hat{v}}{\hat{w}}\cos\theta)\sin(w\cdot dt) + x - \frac{\hat{v}}{\hat{w}}\sin\theta \\ (y - y - \frac{\hat{v}}{\hat{w}}\cos\theta)\sin(w\cdot dt) + (x - x - \frac{\hat{v}}{\hat{w}}\sin\theta)\cos(w\cdot dt) + y + \frac{\hat{v}}{\hat{w}}\cos\theta \\ \theta + \hat{w}dt + \gamma dt \end{bmatrix} \tag{9}$$

$$x_t = \begin{bmatrix} x - \frac{\hat{v}}{\hat{w}}\sin\theta + \frac{\hat{v}}{\hat{w}}\sin(\theta + \hat{w}dt) \\ y + \frac{\hat{v}}{\hat{w}}\cos\theta - \frac{\hat{v}}{\hat{w}}\cos(\theta + \hat{w}dt) \\ \theta + \hat{w}dt + \gamma dt \end{bmatrix} \quad (10)$$

The motion model developed for this assignment takes as input the prior state of the robot, x_{t-1}, the current control velocities, u_t, a time step, dt and returns the posterior state, x_t. Additionally, it utilizes a sampling function (Thurn et. al, 2005, p.98), which samples a normal distribution. Furthermore, the velocities are perturbed by noise parameters: α_1, α_2, α_3, α_4. An additional random term, γ, is computed by sampling the perturbation of the velocities by α_5 and α_6. This velocity motion model resembles the motion model described on page 98 of the Probabilistic Robotics textbook, however, the calculation for the Instantaneous Center of Rotation, as well as the derivation of equation (10), follow the derivation described by Dudek and Jenkin (n.d). Because the motion of this model has dependency on the angular velocity and the heading of the robot, it is nonlinear. The inputs, w and $\boldsymbol{\theta}$, are nonlinear, which means that scaling the linear velocity by the angular velocity, as well as scaling the inputs by sin and cos, result in the output to be nonlinear.

2. Motion Model Command Sequence Results

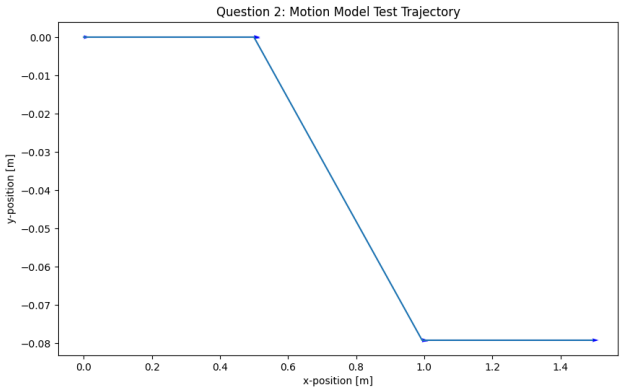


Figure 2: Plot for Step 2 without Noise ($\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha_5 = \alpha_6 = 0$)

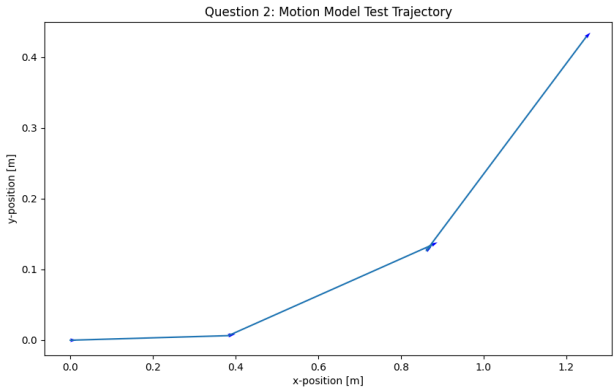


Figure 3: Plot for Step 2 without Noise ($\alpha_1 = \alpha_2 = 0.3 \mid \alpha_3 = \alpha_4 = 0.6 \mid \alpha_5 = \alpha_6 = 1$)

Figure 1 shows that the motion model without any noise performs the sequence of commands precisely, as the robot travels 0.5m, then turns $\frac{-1}{2\pi}$rads, then moves another 0.5m, turns $\frac{1}{2\pi}$rads and finally moves another 0.5m. Figure 3 shows that adding noise, as indicated by the alphas below the figure, makes the robot travel with varying velocities when compared to the input velocities.

3. Motion Model Dataset Results

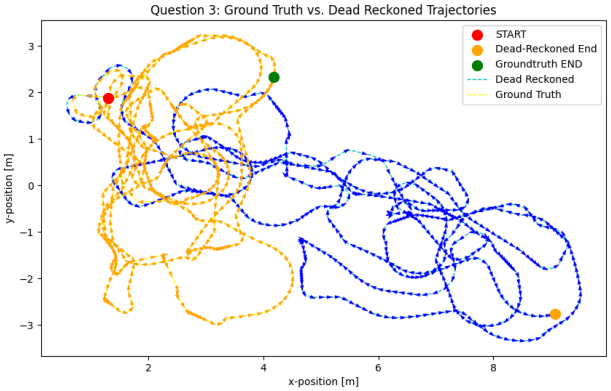


Figure 4: Plot for Dataset without Noise
($\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha_5 = \alpha_6 = 0$)

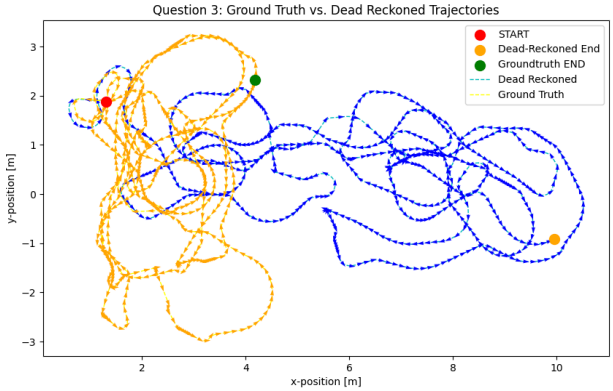


Figure 5: Plot for Dataset with Noise
($\alpha_1 = \alpha_2 = 0.3 \mid \alpha_3 = \alpha_4 = 0.6 \mid \alpha_5 = \alpha_6 = 1$)

The plots seen in Figures 4 and 5 showcase the dead-reckoned path the motion model outputs with the commands from _Control.dat. Figure 4 showcases the motion model without any noise added to the input velocities. Figure 5 showcases the motion model with the noise parameters listed below the plot. From these results it can be seen that both motion models are unable to converge to the ground truth path, and the model with noise diverges more indicated by the fact that its end point is farther along the x-axis. These results suggest that the real robot experienced noise in its motion. This noise is stochastic and could have been caused by the wheels slipping or the motors not commanding the actual number of rotations, resulting in a shorter distance travelled than what should theoretically be commanded given the input commands. Adding noise to the motion model in this step has no effect as only one state is being sampled and no correction due to measurements is being done.

4. The Particle Filter

The particle filter is a non-parametric filter which randomly samples a finite number of states, which are represented as particles. It takes as input a set of M particles, as seen in equation (11), from the prior and a set of control commands.

$$x_{t-1}^{[0:M]} = \{x_{t-1}^{[0]}, x_{t-1}^{[1]}, \dots, x_{t-1}^{[M]}\} \tag{11}$$

The filter then loops through each element in the prior state set, and samples a new state transition which is added to a temporary state transition set of the same size. The sampling is done by passing each particle in the motion model, which results in a particle that is perturbed in accordance with the added noise in the motion model. The motion model computation is proportional to the Bayes filter posterior:

$$x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]}) \tag{12}$$

With the state transition computed, a corresponding weight is assigned to each particle in the set. The weight incorporates the probability of measurement z_t, which results in a set of weighted particles that approximates the Bayes filter posterior (Thurn et. al, 2005, p.78).

$$w_t^{[m]} = p(z_t \mid x_t^{[m]}) \tag{13}$$

In practice for this assignment the weight is computed by obtaining the probability densities for the bearing and range measurements by passing the predicted measurements into a Gaussian distribution centered around the actual measurements to each landmark. The two densities are then multiplied to compute the output weight of each particle.

$$w_{range}^{[m]} = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (14)$$

$$w_{bearing}^{[m]} = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (15)$$

$$w^{[m]} = w_{range}^{[m]} * w_{bearing}^{[m]} \quad (16)$$

Each particle and weight is then added to the state transition set, and the weights of each particle are normalized.

$$w = \frac{\{w_t^{[0]}, w_t^{[1]}, \dots, w_t^{[M]}\}}{\sum_{i-0}^{M} w_t^{[i]}} \quad (17)$$

The particles are then resampled proportionally to their associated weight. A major assumption that forms the basis of the Particle Filter is that the filter follows the Markov process, which assumes that each state transition and measurement is independent of previous measurements and controls (Thurn et. al, 2005, p.29). Despite this, however, particle filters are robust and effective due to multi-modal representation of the state at each timestep. This filter is also capable of handling nonlinear inputs, which Gaussian filters like the basic Kalman filter are incapable of. Furthermore, the accuracy of the model scales in proportion with the number of particles in the system, however, this comes at a major computational cost. Lastly, because of the constant particle resampling, particles with poor predictions are sampled out, which allows the system to create new predictions that can lead to convergence.

5. Measurement Model Design

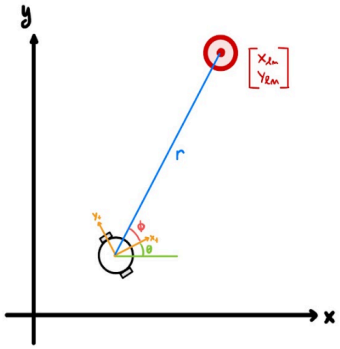


Figure 6: Range (r) and Bearing (ϕ) Visualization for Measurement Model

$$r = \sqrt{(x_t - x_{lm})^2 + (y_t - y_{lm})^2} \quad (18)$$

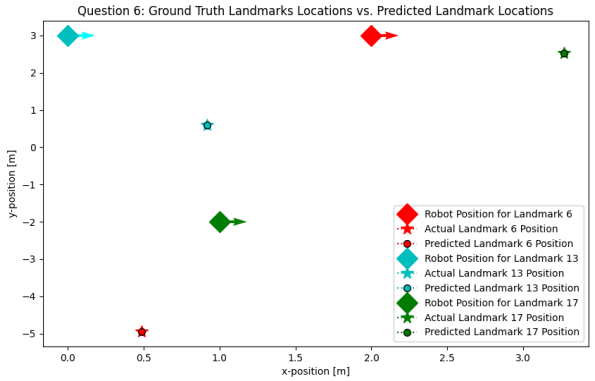
$$tan(\theta + \phi) = \frac{y_t - y_{lm}}{x_t - x_{lm}} \quad (19)$$

$$\phi = tan^{-1}\left(\frac{y_t - y_{lm}}{x_t - x_{lm}}\right) - \theta \quad (20)$$

The range between the robot and the landmark is the Euclidean distance between the two objects, while the bearing is determined by taking the difference between the angle determined from the arctangent

computation and the heading of the robot. Because there is no inherent noise added in any of the computations, the measurement model is able to precisely determine the range and bearing values.

6. Measurement Model Test




```
Predicted Distance to Landmark 6: 8.0939 [m]
Predicted Bearing to Landmark 6: -1.7588 [rad]
Predicted Position of Landmark 6: (0.487, -4.9513) [m]
Actual Position of Landmark 6: (0.487, -4.9513) [m]
Percent Error between Actual and Predicted Landmark 6 Position: (0.0, -0.0)

Predicted Distance to Landmark 13: 2.5729 [m]
Predicted Bearing to Landmark 13: -1.2061 [rad]
Predicted Position of Landmark 13: (0.9177, 0.5963) [m]
Actual Position of Landmark 13: (0.9177, 0.5963) [m]
Percent Error between Actual and Predicted Landmark 13 Position: (-0.0, 0.0)

Predicted Distance to Landmark 17: 5.0633 [m]
Predicted Bearing to Landmark 17: 1.1065 [rad]
Predicted Position of Landmark 17: (3.2673, 2.5273) [m]
Actual Position of Landmark 13: (3.2673, 2.5273) [m]
Percent Error between Actual and Predicted Landmark 17 Position: (0.0, -0.0)
```

Figure 7: Plot for Step 6 Location and Landmark Table (left) and Predicted Values (right)

The print out to the terminal shows the computed range and bearing for each position and landmark list in step 6. These computed ranges and bearings were used to predict the position of the landmark, and then checked to see if there was any discrepancy. From the print out, and the plot, it can be seen that the predicted range and bearing yield correct predicted positions.

7. Particle Filter Implementation

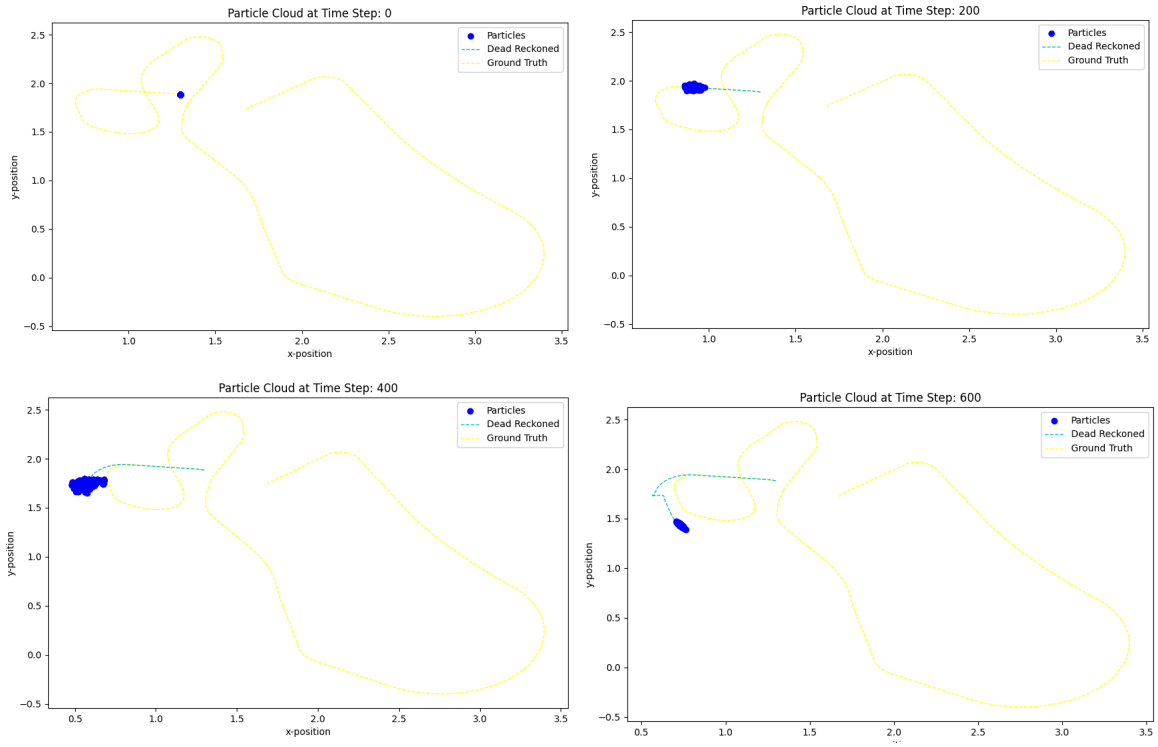


Figure 8: Evolution of Particle Cloud for a Snippet of the Dataset Trajectory (500 Particles)

Figure 8 demonstrates that the particle cloud, representing different robot states, expands over time, until the uncertainty of the states decreases, at which point the particle cloud begins to compress. The figure

showcases the particle cloud at timestep zero (top-left) to the particle cloud at timestep 600 (bottom-right). The particle cloud starts off as a point mass, and slowly expands as the uncertainty of the state increases.

8. Particle Filter Performance Comparison

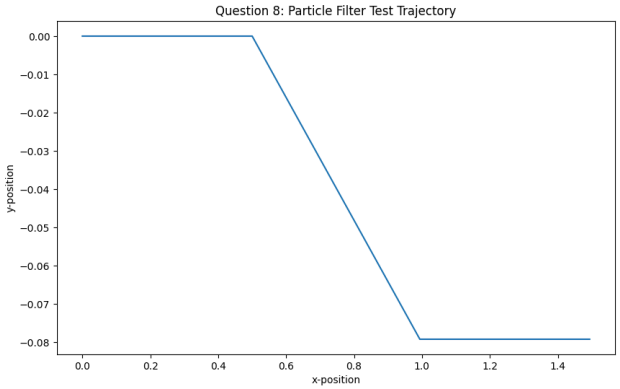


Figure 9: Plot for Step 2 without Noise for PF
($\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha_5 = \alpha_6 = 0$)

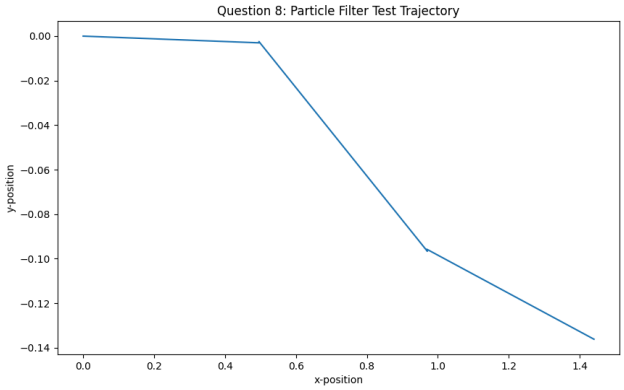


Figure 10: Plot for Step 2 with Noise for PF
($\alpha_1 = \alpha_2 = 0.3 \mid \alpha_3 = \alpha_4 = 0.6 \mid \alpha_5 = \alpha_6 = 1$)

Figure 9 shows that without any noise in the motion model, the Particle Filter functions exactly the same as the motion model without any noise. Figure 10, on the other hand, shows that even with noise perturbation, the Particle Filter attempts to converge to the expected path of the unperturbed motion model. This result is interesting because there are no measurements correcting the posterior, and yet the filter remains in similar range values for the output. The performance of the filter is not perfect in this situation because of the lack of those measurements, however, it performs a lot better than the motion model with noise.

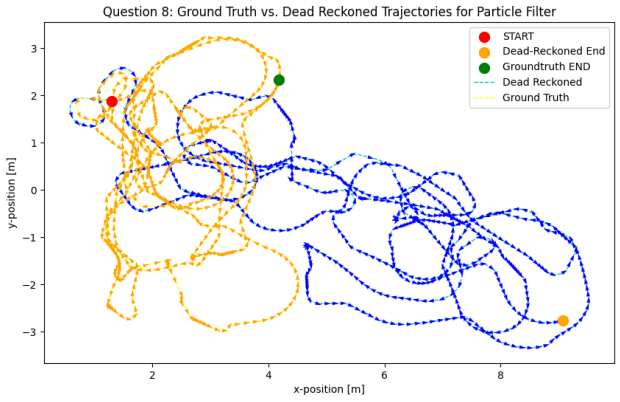


Figure 11: Plot for Dataset without Noise for PF
($\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha_5 = \alpha_6 = 0$)

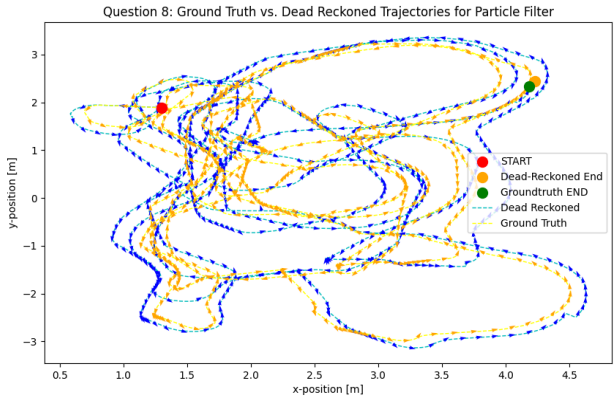


Figure 12: Plot for Dataset with Noise for PF
($\alpha_1 = \alpha_2 = 0.3 \mid \alpha_3 = \alpha_4 = 0.6 \mid \alpha_5 = \alpha_6 = 1$)

The following plots were generated with a particle filter whose particle number was 100 with a standard deviation of 0.02 for the bearing Gaussian distribution and a standard deviation of 0.1 for the range Gaussian Distribution. Similar to Figure 9, Figure 11 shows the trajectory of the filter without any noise in the system. This means that the particle cloud is acting as point mass, and is therefore behaving in the same way as the simple motion model. In this particular case, the Particle Filter performs poorly due to the fact that there is no distribution of particles to sample new/different states from. Figure 12 shows the trajectory of the filter with the noise parameters listed below the figure. The performance of the filter in this plot is really

good, as the dead-reckoned trajectory looks to always be within a few centimeters of the ground truth trajectory. Furthermore, the distance between the two endpoints is 0.157m. The reason for this improved performance is because the particle cloud is able to distribute in proportion to the noise parameters, which generate many different states thanks to equation (3).

9. Particle Filter Uncertainty Analysis

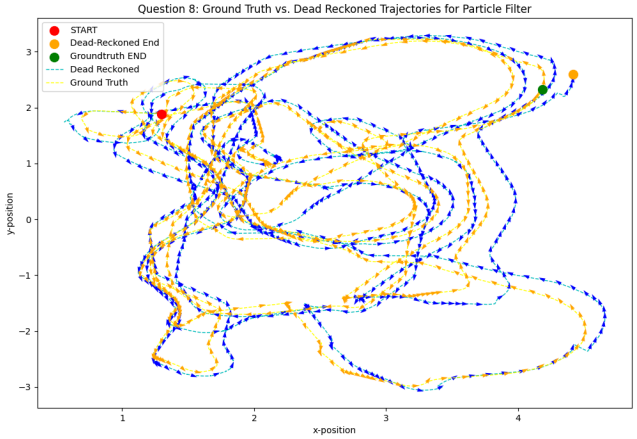


Figure 13: Plot for Dataset with Noise for PF ($\alpha_1 = \alpha_2 = 1 \mid \alpha_3 = \alpha_4 = 2 \mid \alpha_5 = \alpha_6 = 1$)

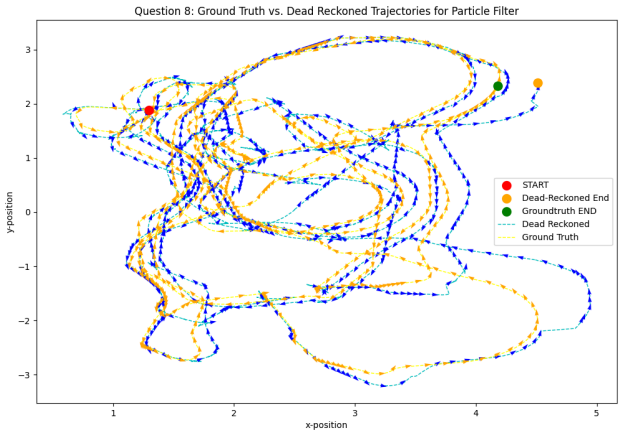


Figure 14: Plot for Dataset with Noise for PF ($\alpha_1 = \alpha_2 = 3 \mid \alpha_3 = \alpha_4 = 5 \mid \alpha_5 = \alpha_6 = 4$)

The following plots varied the amount of noise added in the motion model, but maintained the standard deviation of 0.02 for the bearing Gaussian distribution and a standard deviation of 0.1 for the range Gaussian Distribution. Figure 13 added slightly more noise than the best performing plot from the previous experiments, and as such experienced an endpoint distance of 0.394m. Figure 14 added a larger amount of noise into the system, which resulted in an endpoint distance of 0.371m. This result suggests that the amount of noise has some impact on the performance of the particle filter, however, when analyzing the figures more closely, it can be seen that adding a large amount of noise causes the trajectory to diverge a lot more overall. There is also more noise in the trajectory.

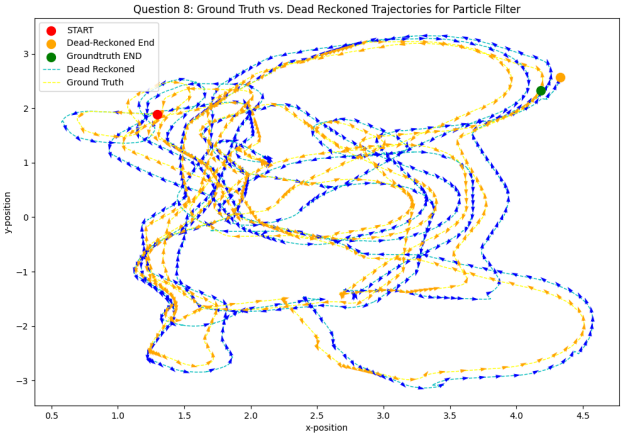


Figure 15: Plot for Dataset with Noise for PF ($\alpha_1 = \alpha_2 = 1 \mid \alpha_3 = \alpha_4 = 1 \mid \alpha_5 = \alpha_6 = 1$)

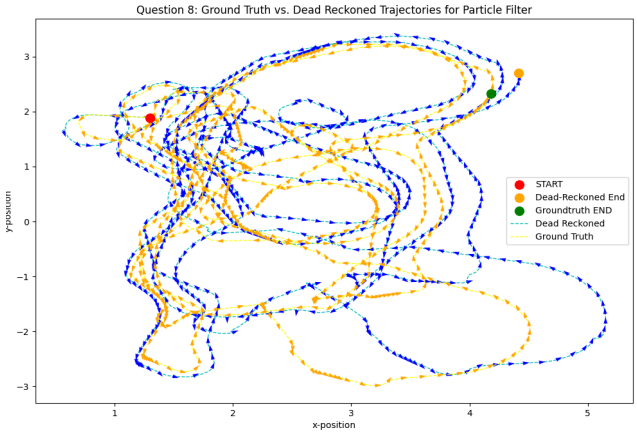


Figure 16: Plot for Dataset with Noise for PF ($\alpha_1 = \alpha_2 = 1 \mid \alpha_3 = \alpha_4 = 1 \mid \alpha_5 = \alpha_6 = 1$)

The following plots varied the amount of standard deviation (0.7 and 0.5) for Figure 15 and (0.1 and 0.01) for Figure 16, but maintained the noise. Figure 15 had an endpoint distance of 0.280m and Figure 16

had an endpoint distance of 0.439m. This result suggests that the amount of noise has some impact on the performance of the particle filter, however, when analyzing the figures more closely, it can be seen that adding a large amount of noise causes the trajectory to diverge a lot more overall. There is also more noise in the trajectory.

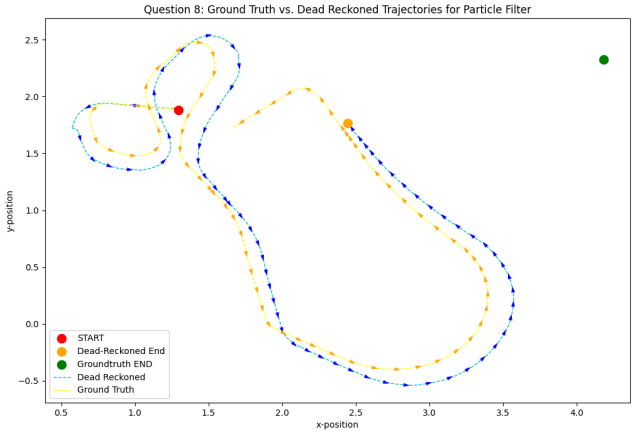


Figure 17: No Missing Measurement
($\alpha_1 = \alpha_2 = 0.3$ | $\alpha_3 = \alpha_4 = 0.6$ | $\alpha_5 = \alpha_6 = 1$)

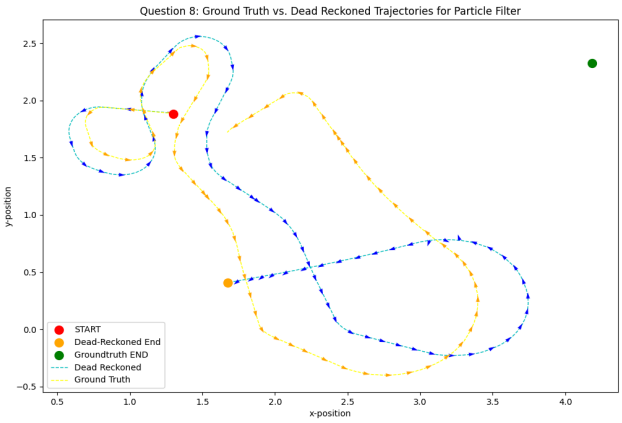


Figure 18: Missing Measurement
($\alpha_1 = \alpha_2 = 0.3$ | $\alpha_3 = \alpha_4 = 0.6$ | $\alpha_5 = \alpha_6 = 1$)

The following plots explored one sample of the trajectories where the measurements were present, and another sample where measurements were not present. Figure 17 shows the case for which the measurements were present, and it can be seen that the Particle Filter converges. Figure 18 showcases the case for which the measurements were not present, and it can be seen that the filter diverges.

References Cited

Thrun, S., Fox, D., & Burgard, W. (2005). Probabilistic Robotics (intelligent robotics and autonomous agents series): 9780262201629. https://www.amazon.com/Probabilistic-Robotics-INTELLIGENT-ROBOTICS-AUTONOMOUS/dp/0262201623

Russell, S., & Norvig, P. (2020.). Artificial Intelligence: A modern approach, 4th us ed. Artificial Intelligence: A Modern Approach, 4th US ed. https://aima.cs.berkeley.edu/

Dudek, & Jenkin. (n.d.). CS W4733 notes - Differential Drive Robots. https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf